

Incremental Data Load in Azure Data Factory

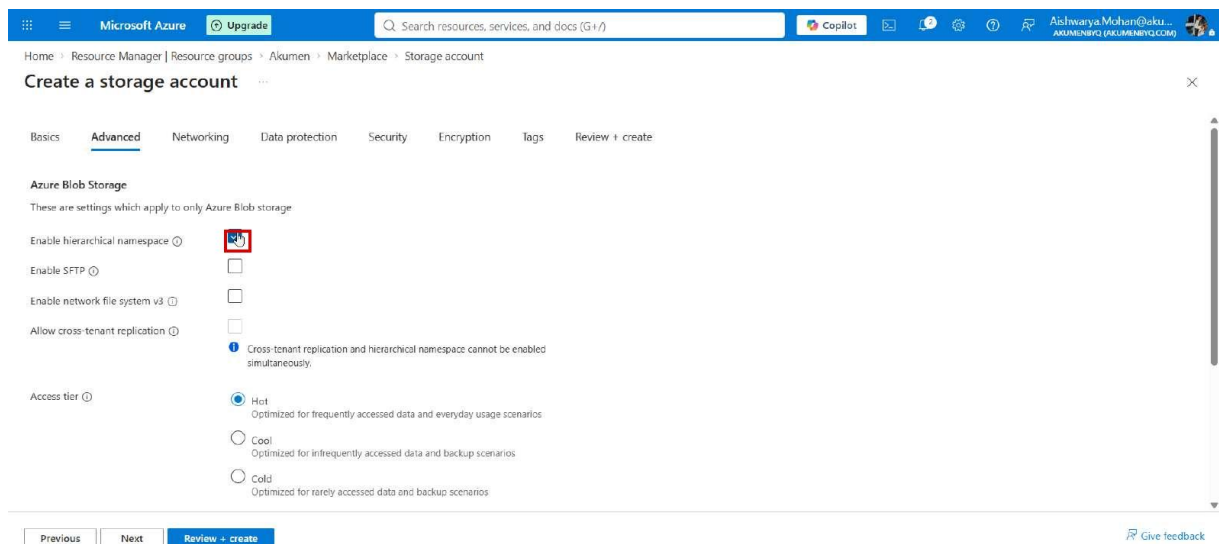
Using the watermark method | Aishwarya Mohan

Following on from the basic Copy Activity pipeline, this one loads only the **new or changed rows** from a SQL table instead of copying the whole table every single run. The idea is called the **watermark method** — I keep a small table that stores the last timestamp that was loaded. Every run, the pipeline checks that stored value, pulls only the rows added after it, copies them out, and then updates the stored value for next time.

1. Setting up the Data Lake (ADLS Gen2)

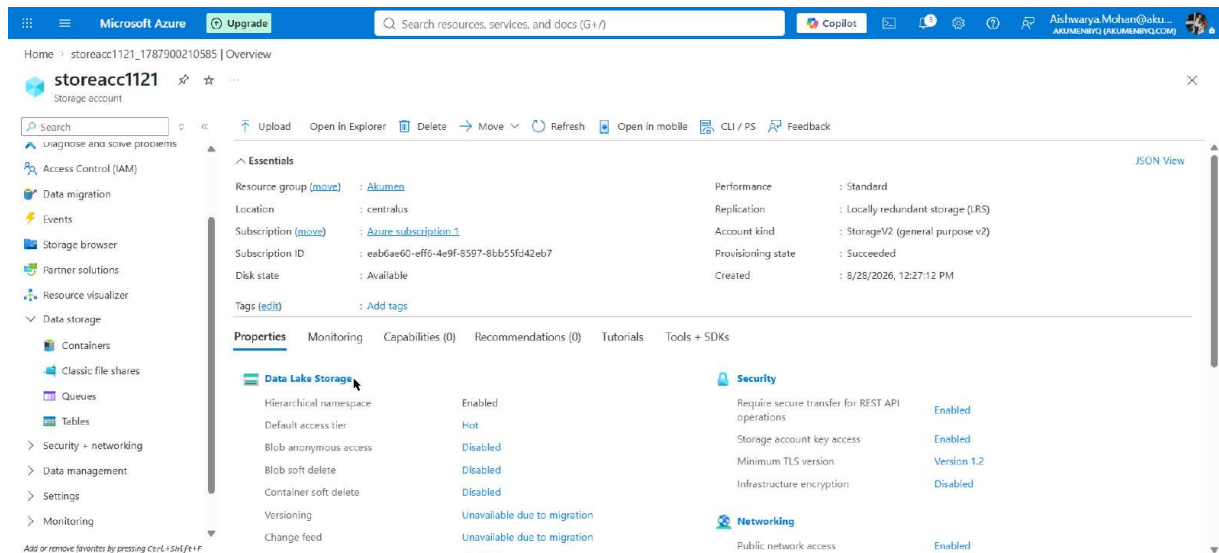
1.1 Create the storage account with hierarchical namespace on

This is the setting that actually turns a normal storage account into a Data Lake Gen2 account. Easy to miss because it's tucked away in the Advanced tab.



1.2 Confirmed it went through

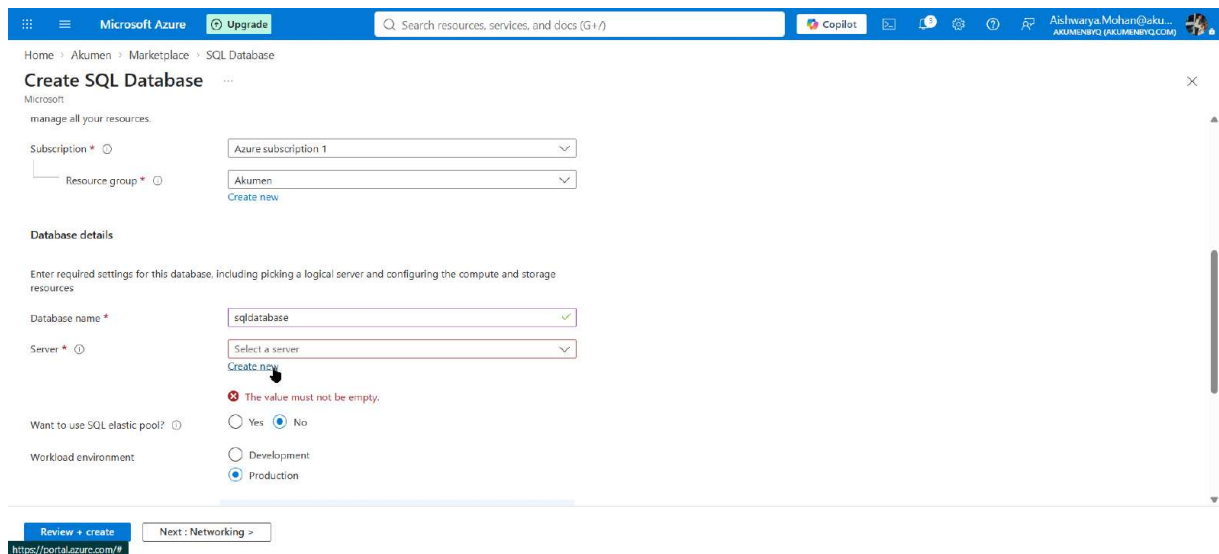
Overview page shows Hierarchical namespace as Enabled, so this is a proper ADLS Gen2 account and not just plain blob storage.



2. Setting up the Azure SQL Database

2.1 Create the SQL database

Named it sqldatabase, put it in the same Akumen resource group as everything else.



2.2 Create a new logical server for it

Went with SQL authentication (username/password) instead of Entra-only, just to keep things simple for this exercise.

Home > Akumen > Marketplace > SQL Database > Create SQL Database

Create SQL Database Server

Microsoft

Server name * .database.windows.net

Location *

Authentication

1 Azure Active Directory (Azure AD) is now Microsoft Entra ID. [Learn more](#)

Select your preferred authentication methods for accessing this server. We recommend using only Microsoft Entra authentication. [Learn more](#) or using an existing Microsoft Entra user, group, or application as Microsoft Entra admin. [Learn more](#)

Authentication method

- ☐ Use Microsoft Entra-only authentication
- ☐ Use both SQL and Microsoft Entra authentication
- ☒ Use SQL authentication

Server admin login *

Password *

OK

[Feedback](#)

2.3 Picked compute + storage, kept it minimal

General Purpose tier, 2 vCores, 16 GB storage, locally-redundant backups. This is just a practice database so no need to go bigger.

Home > Akumen > Marketplace > SQL Database

Create SQL Database

Microsoft

☒ Production

1 Default settings provided for Production workloads. Configurations can be modified as needed.

Compute + storage * ☐

General Purpose
Standard-series (Gen5), 2 vCores, 16 GB storage, zone redundant disabled
[Configure database](#)

Backup storage redundancy

Choose how your PITR and LTR backups are replicated. Geo restore or ability to recover from regional outage is only available when geo-redundant storage is selected.

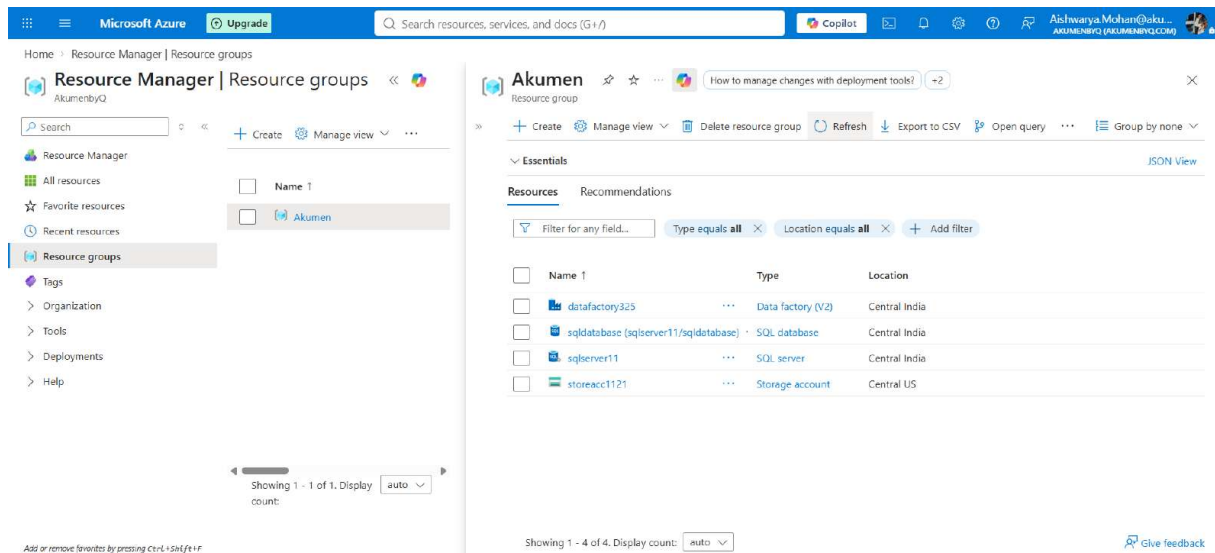
Backup storage redundancy ☐

- ☒ Locally-redundant backup storage
- ☐ Zone-redundant backup storage
- ☐ Geo-redundant backup storage
- ☐ Geo-Zone-redundant backup storage

Review + create **Next: Networking >**

2.4 Everything shows up in the resource group

Storage account, SQL server, SQL database, and the Data Factory — all four resources I need for this exercise.

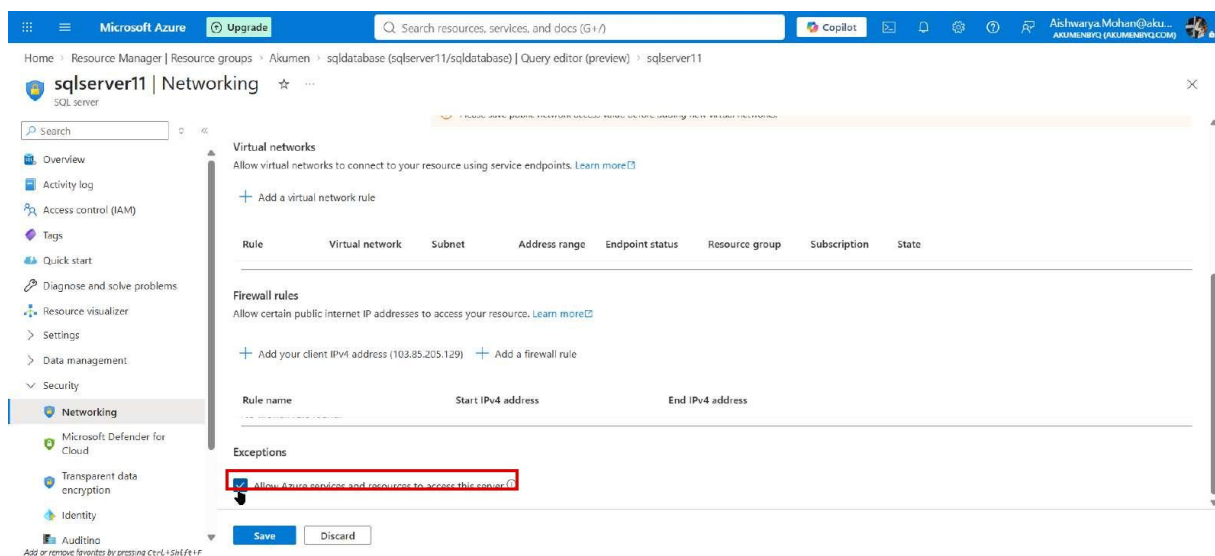


3. Getting into the database and setting up sample data

3.1 Opened up networking so I could actually connect

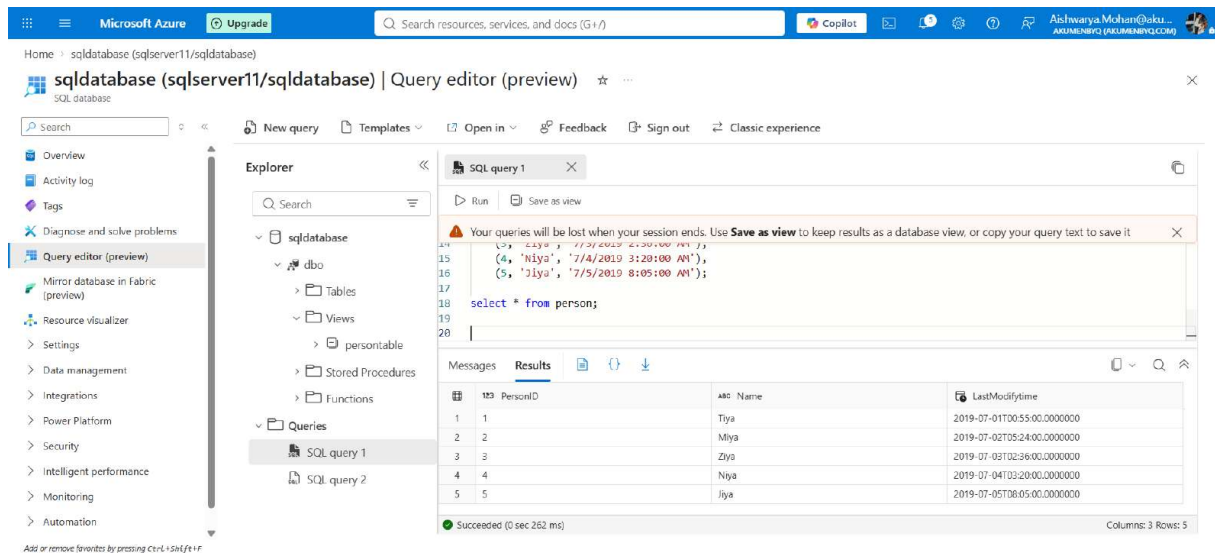
Added my client IP under firewall rules, and also ticked 'Allow Azure services and resources to access this server'.

Without that checkbox, Data Factory itself would get blocked from talking to the SQL server later — this tripped me up on the first Copy Activity project too, so made sure to turn it on early this time.



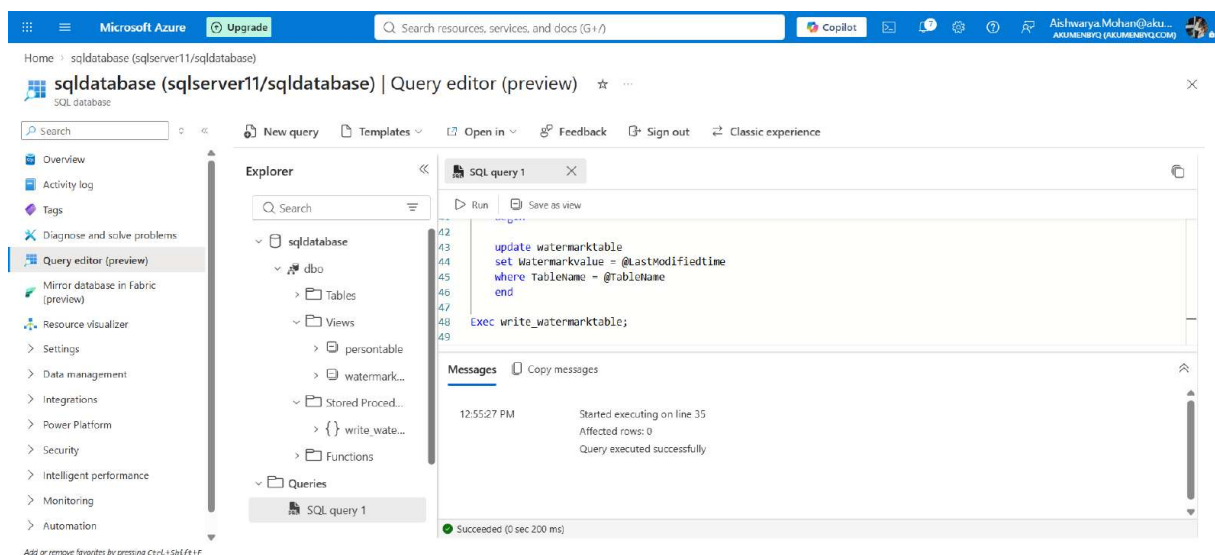
3.2 Sample table already had some data in it

Ran a quick select * on the person table — 5 rows, each with a LastModifytime column. That timestamp column is what the whole incremental logic hangs off.



3.3 Created a watermark table + a stored procedure to update it

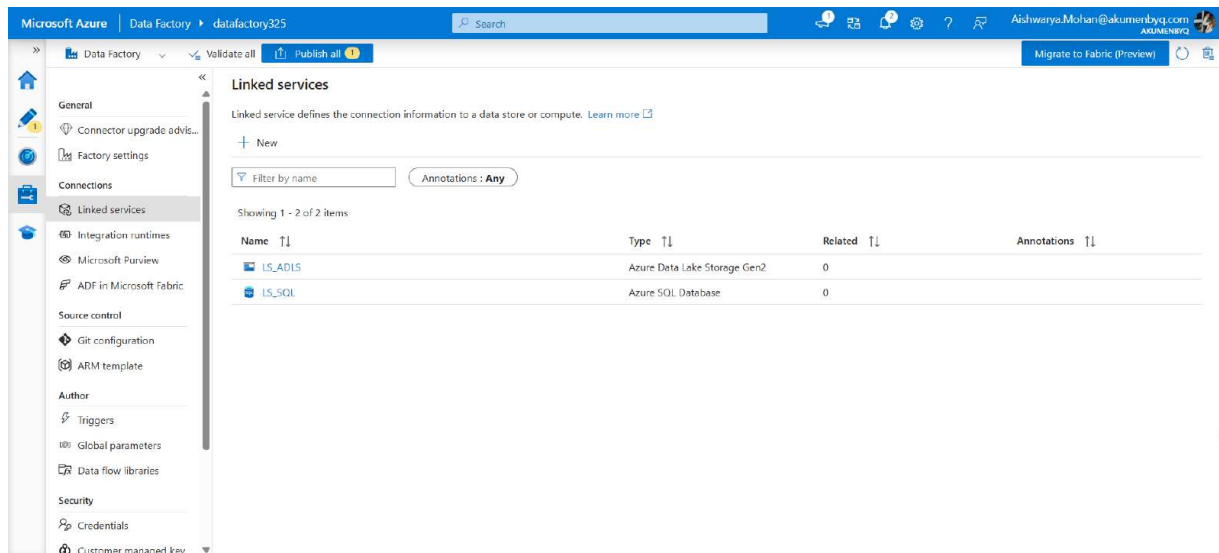
watermarktable just stores one row per source table with its last-loaded timestamp. write_watermarktable is a small stored procedure that updates that value — ran it once here to make sure it compiles and works before wiring it into the pipeline.



4. Building the pipeline in Data Factory

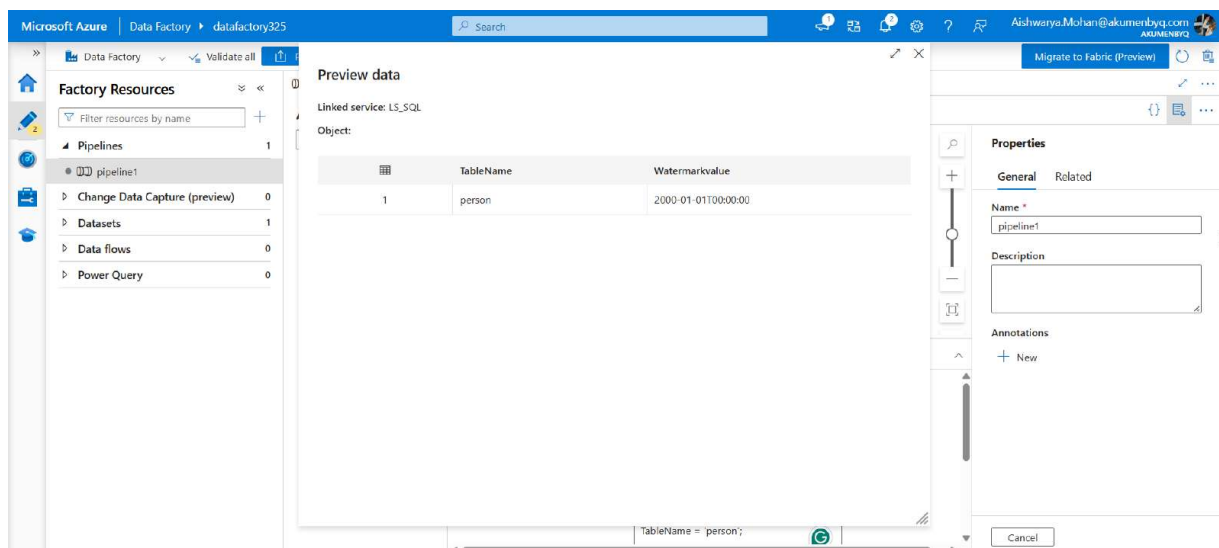
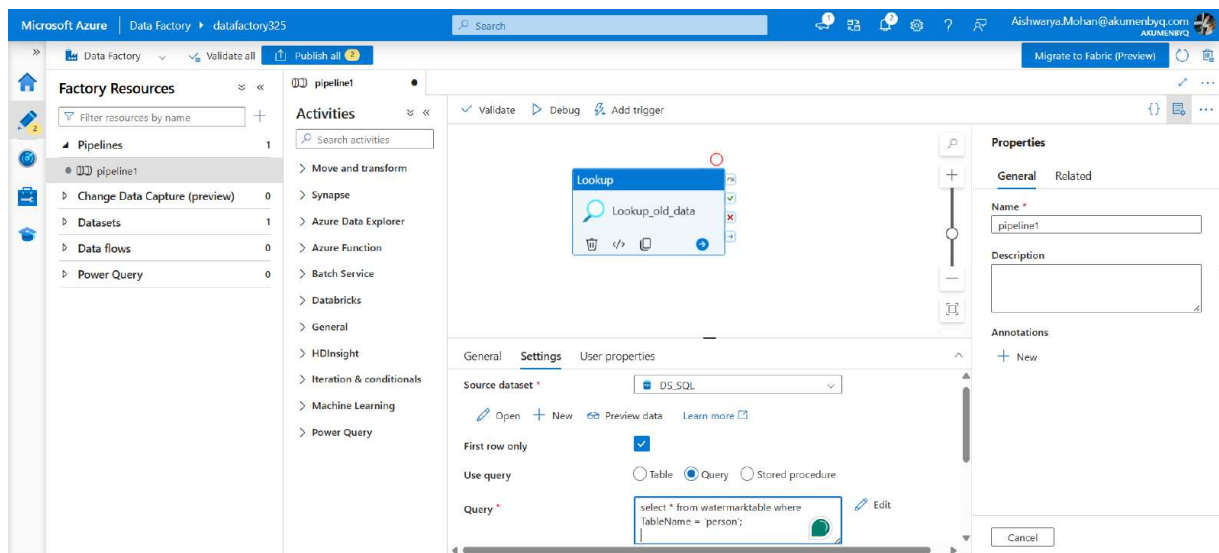
4.1 Linked services to both ends

LS_SQL connects to the Azure SQL Database, LS_ADLS connects to the Data Lake. Both get reused across all the activities below.



4.2 Lookup #1 — get the old watermark

Lookup_old_data runs `select * from watermarktable where TableName = 'person'` and grabs whatever watermark value is currently stored. Previewed it and got 2000-01-01 — makes sense, since this table has never actually been loaded before, that's just the placeholder default.



4.3 Lookup #2 — get the new watermark

Lookup_new_data runs `select max(LastModifytime) as NewWatermarkvalue from person` to find the latest timestamp actually sitting in the source table right now. Preview came back with 2019-07-05 08:05 — that's the newest row in the sample data.

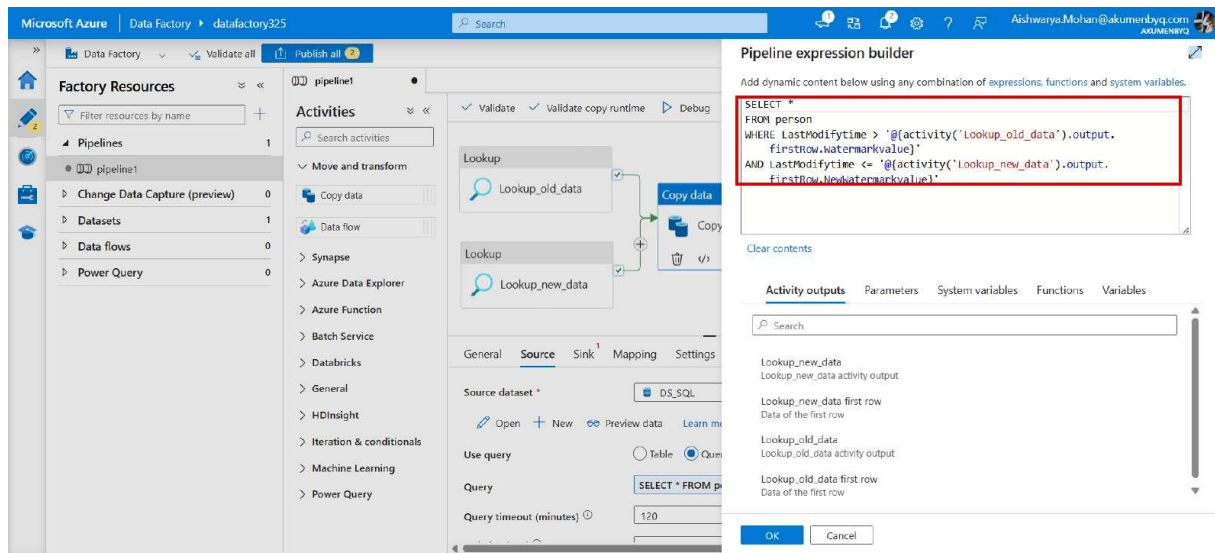
The top screenshot shows the Microsoft Azure Data Factory interface. The 'Lookup' activity is selected, and the 'Preview data' button is highlighted. The query is `select max(LastModifytime) as NewWatermarkvalue from person`.

The bottom screenshot shows the 'Preview data' window. The linked service is 'LS_SQL' and the object is 'NewWatermarkvalue'. The preview shows a single row with the value '2019-07-05T08:05:00'.

	NewWatermarkvalue
1	2019-07-05T08:05:00

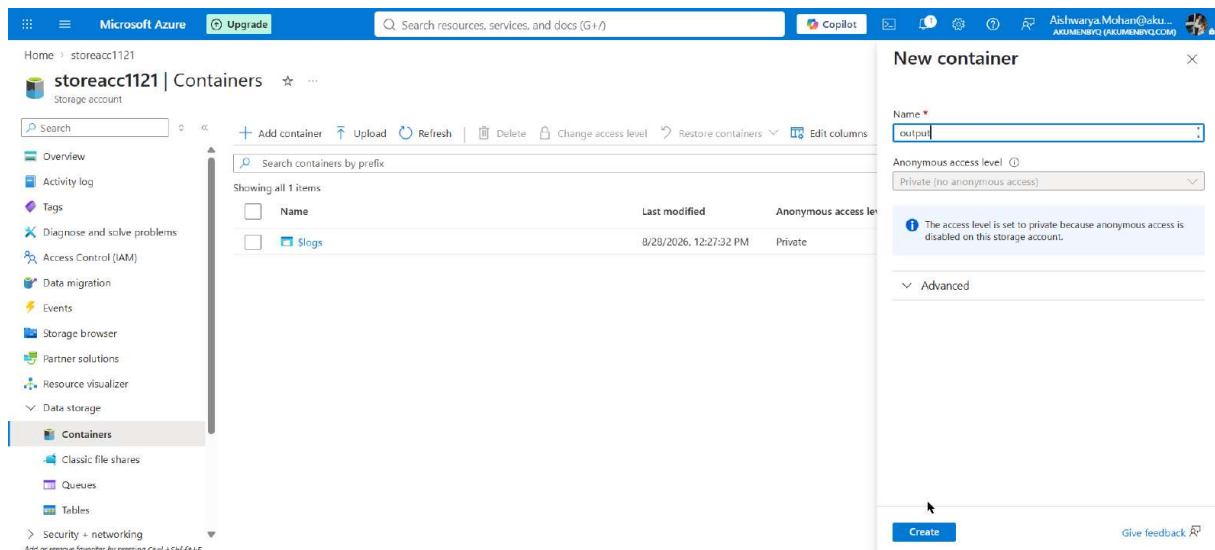
4.4 Copy Data — pull only what's between the two watermarks

The source query for the Copy activity is built dynamically, pulling both lookup outputs in: rows where LastModifytime is greater than the old watermark and less than or equal to the new one. First run, that's effectively 'everything', since the old watermark is still the 2000-01-01 default.



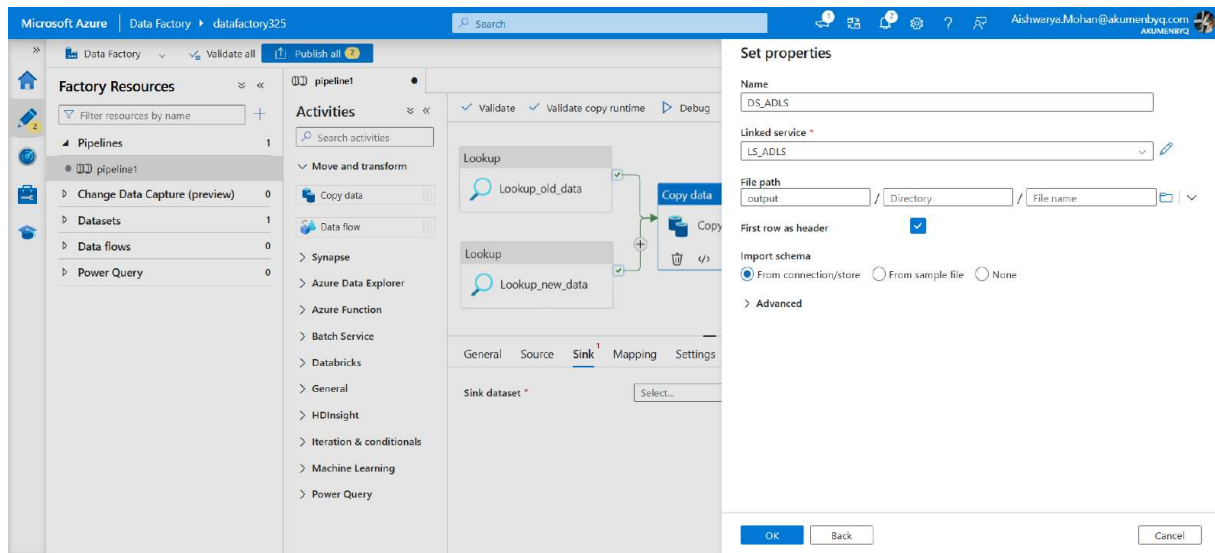
4.5 Made a landing spot in the Data Lake

Added a container called output in the storage account for the copied file to land in.



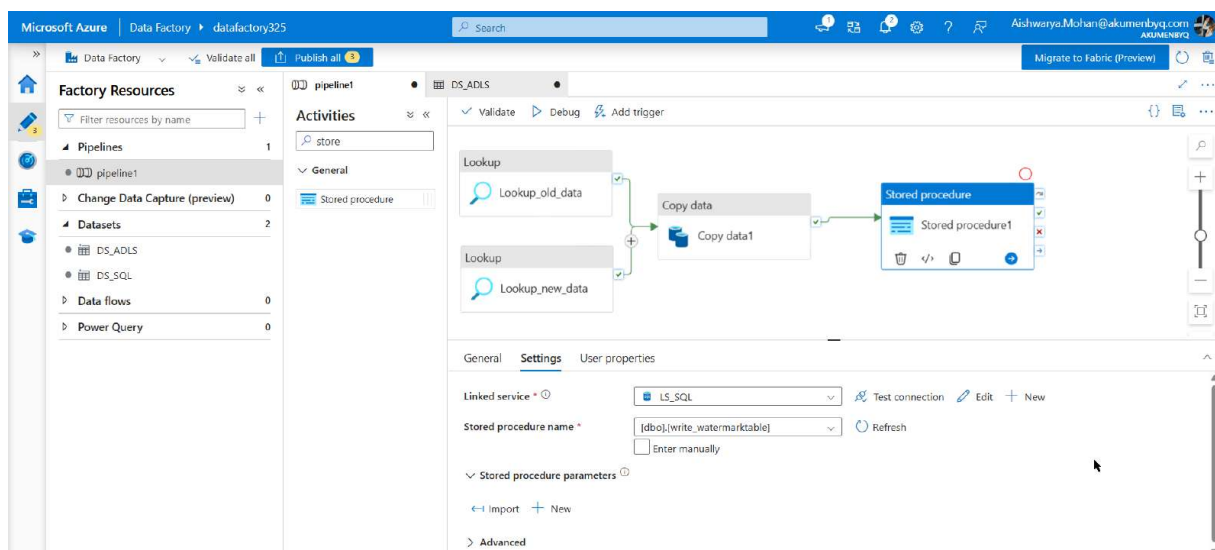
4.6 Pointed the sink at that container

Sink dataset DS_ADLS uses the LS_ADLS linked service, file path set to output, written out as CSV with the first row as header.



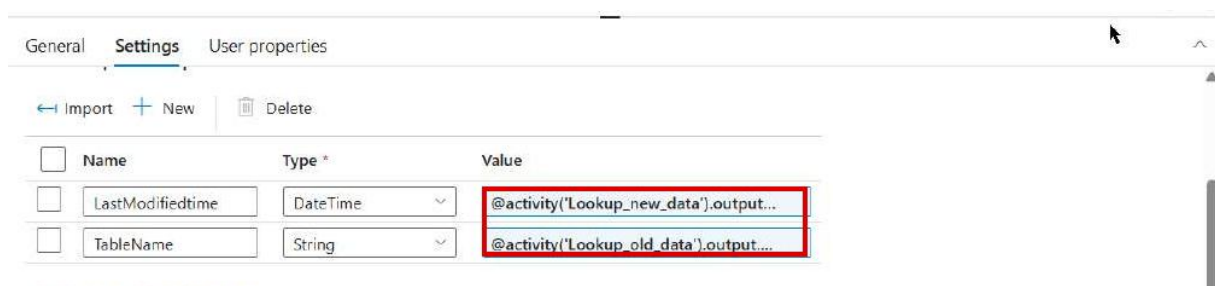
4.7 Stored Procedure activity — update the watermark for next time

Added this right after Copy Data, calling the same write_watermarktable procedure from earlier. This is what actually moves the watermark forward so the next run doesn't reload the same rows again.



4.8 Wired the new watermark into the stored procedure's parameters

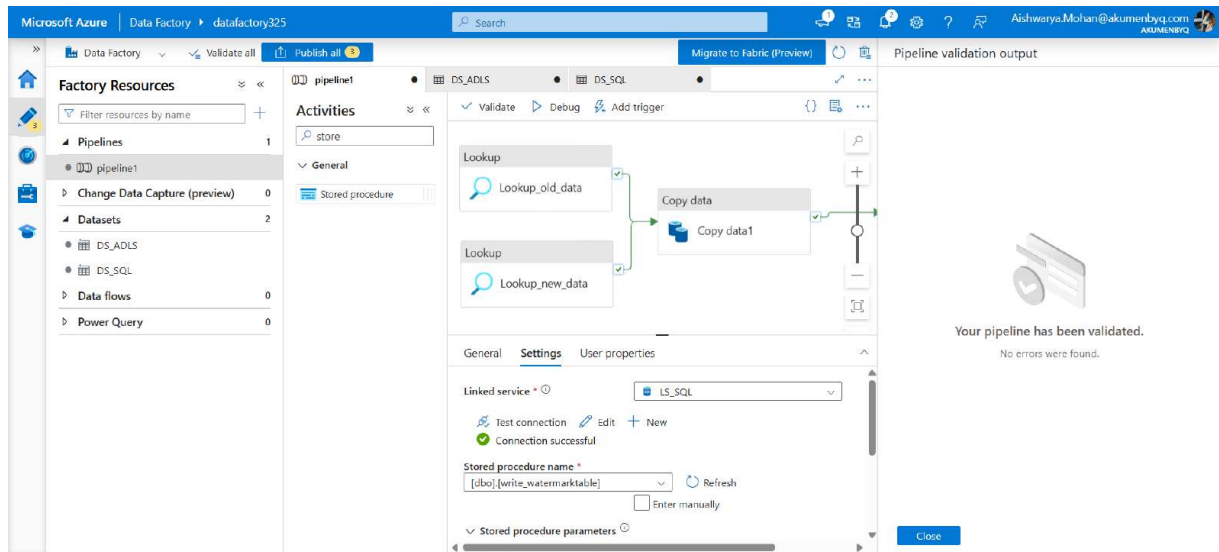
LastModifiedtime is set to Lookup_new_data's output, TableName is set to Lookup_old_data's output — both dynamic expressions again, no hardcoding.



5. Running it and checking the output

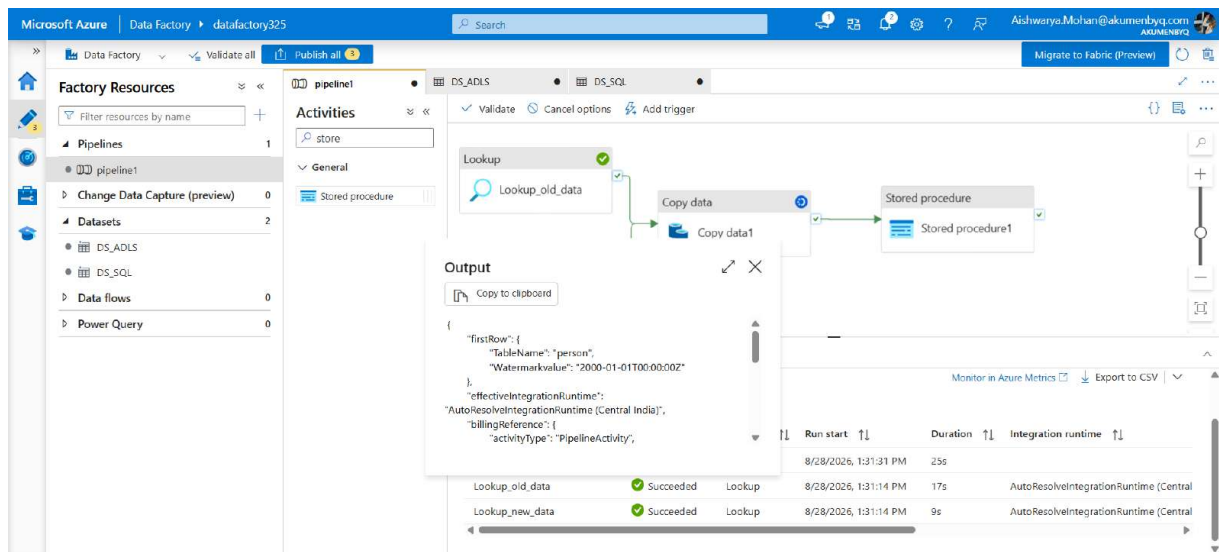
5.1 Validated first

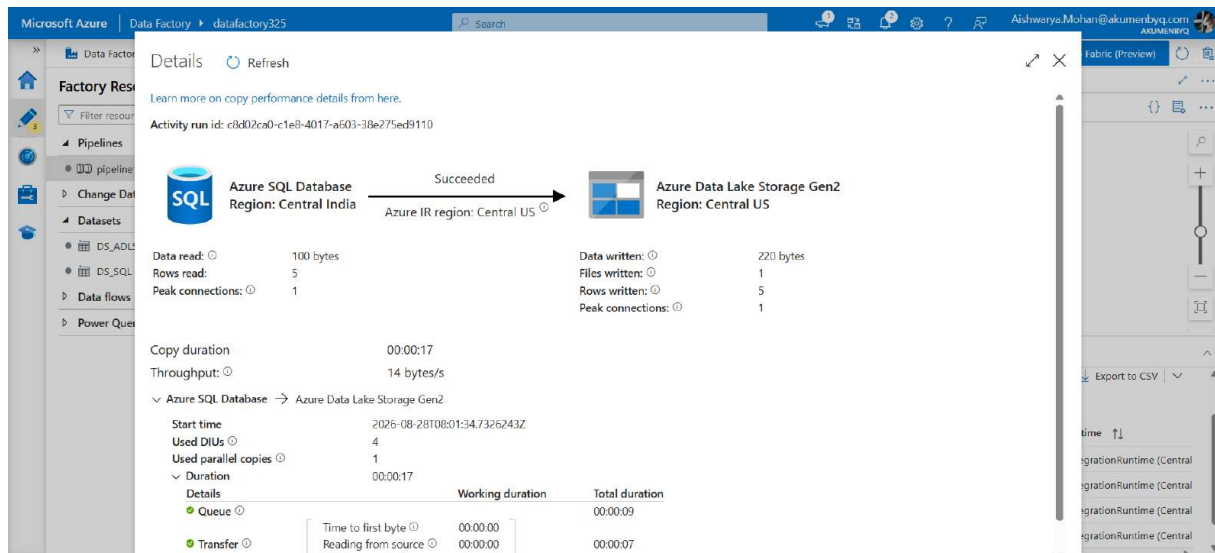
No errors, so all four activities are wired up correctly.



5.2 Debug run

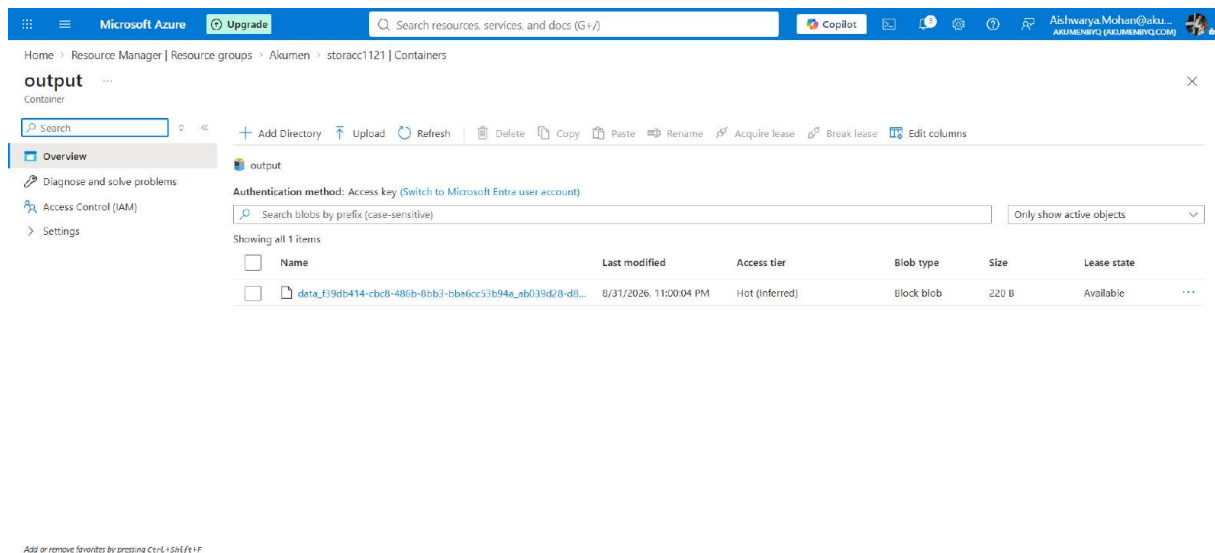
Both lookups succeeded and picked up the values expected — the old watermark output shows the 2000-01-01 default. The Copy Data activity details confirm 5 rows read from SQL and 5 rows written out to the Data Lake, so nothing got dropped along the way.





5.3 File landed in the output container

One CSV file sitting in the output container after the run.



5.4 Opened it up to double check

Same 5 rows as the source table (Tiya, Miya, Ziya, Niya, Jiya) with their timestamps intact, so the copy went through correctly.

The screenshot shows the Microsoft Azure portal interface. On the left, the 'output' container is selected under the 'Blob' service. The main pane displays the 'data_f39db414-cbc8-486b-8bb3-bba6cc53b94a_ab039d28-d805-49fd-a88d-7e81fa272a26.csv' file. The file is shown in a table view with the following data:

PersonID	Name	LastModifiedTime
1	"Tiya"	2019-07-01 00:55:00.0000000
2	"Niya"	2019-07-02 05:24:00.0000000
3	"Ziya"	2019-07-03 02:36:00.0000000
4	"Niya"	2019-07-04 03:20:00.0000000
5	"jiya"	2019-07-05 08:05:00.0000000

On the next run, `Lookup_old_data` would pick up 2019-07-05 08:05 instead of the 2000-01-01 default, so the pipeline would only pull rows added or changed after that point — not the same 5 rows all over again. That's the whole point of storing the watermark instead of just copying the full table every time.